ALGORITHM SHORT NOTE



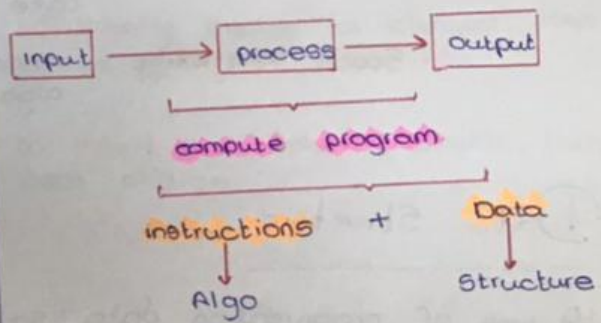
Algo short note.

Algorithms are step-by-step procedure for solving problems

- Flow chart
- pseudo code

many deferent ways in real world

- sort data
- Search for infomation
- Navigate through website
- Recomend product to customer
- make decision
- traffic control



Characteristics

- Finitness - finit number of step
- Unambiguty - clear and easy to understand
- Effectiveness - must be basic, meaning that it can be
- Input / output - zero or more inputs and at lecst one output.
- Correctness.

Type of Algorithms

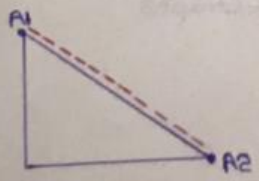
01. Sorting Algorithm : used to arrange a list of items in specific oder such as ascending or descending

5 3 1 2 4 —sort(asc)→ 1 2 3 4 5

02. Searching Algorithm : used to find a specific item in a list or other data structure.

Find 5. → 1 2 3 4 [5] 6 7 → found 5

03. Graph Algorithm : used to perform oparation on graph such as finding the shortest path between two nodes



04. String Algorithm : uset to perform oparation on string such as finding longest common substring

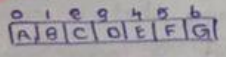
Onece an algorithms has been deeigned, it is important to analize its performance.

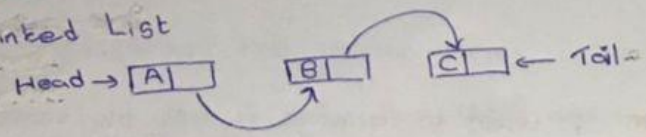
This includes analizing the algorithms time complexity and space complexity.

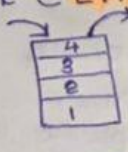
- **Time complexity** - measure of the amt of time that algo ~~teat~~ take to run.
- **Space complexity** - measure of the amt of memory that the algo requires to run.

Data Structure

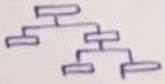
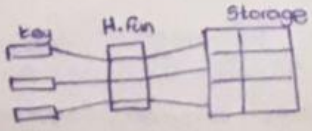
A way of organizating data so that it can be used efficiently.
main purpose is to write efficient program.

- Linear
  - Derect Access :
    - *Array 

print (Array[4])
  - Sequential Access
    - *Linked List 

    - *Stack (LIFO) 

    - *Queue (FIFO) 

- Non- Linear
  - *Tree 
  - *Hash Table 

  - *Graphs

Choosing right data structure

The choice of which data structure to use depends on the specific need of the application.

If application needs to be able to quickly access and modify individual elements, then an array may be the best choice.

If the application need to be able to efficiently seach for element, then a hash table or tree may be the best choice.

If the application needs to be able to insert and delete elements, then a linked list or queue may be the best choice.

Data Structure Applications

- Operating System : manage memory, processes, and other resources
- Compilers : Parse and analize code
- Data Base : store and manage data
- Networking : Network protocol transmit and receive data.
- Graphics : represent and render image.
- Artificial Interlligence : algorithms store and manipulate knowladge

When choosing a data structure important to consider.

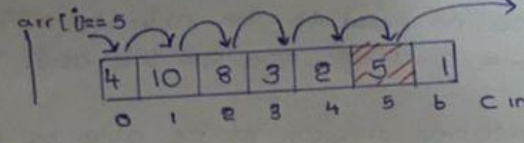
01. Time complexity
02. Space Complexity
03. Easy of use.

Searching Algorithms

01. Liner search Algorithm.

Its works by comparing the target value to each element in the collection unit it finds a match

Target Value : 5




```
list = [ 4, 10, 8, 3, 2, 5, 1 ]
Found = 0

For Ci in rangeClenClist):
    if Ci== 5):
        printC" Found value")
        Found = 1
        breat

IF CFound ==0):
    print C" Not found value")
```

- Easy to understand and implement
- But very slow for big data collod

Applications

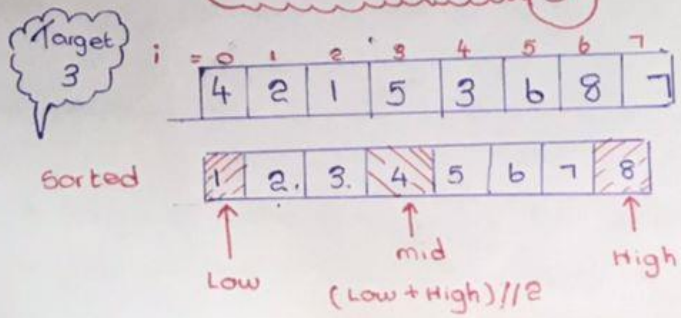
- Small derectory file
- small database

No need to sorted array

02. Binary Search Algorithm

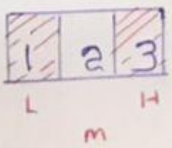
That devides and eliminates half of the search interval with each comparison until the target value is found or the search interval is empty.

Must sorted array



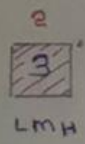
- mid == target
  4 3 NO
- mid < target NO

low = 0
High = mid - 1



- mid == target | NO
  2 3
- mid < target | YES

Low = mid + 1



• mid == target | Yes Found in index No. 2

If it couldn't find output should be -1

```
def bin_s (array, target):
    low = 0
    High = len(array) - 1

    while (low <= High):
        mid = (low + High) // 2

        if (mid == target):
            return mid
        elif (mid < target):
            low = mid + 1
        else:
            High = mid - 1

    return -1
```

```
arr = [4, 2, 1, 5, 3, 6, 8, 7]

print(bin_s(arr, 3))
```

Output : 2

Advantage

* Very efficient with a time complexcity of O(log n).
* Easy to emplement
* Can be used to solve a variety of problems.

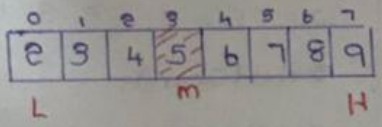
Disadvantage

* Requires the array to be sorted
* Can be less efficient than other searching algo for small array.

Time complexity Analysis of Binary Search Algo

01. Best Case

* minimum NO. of comparision
* Search element is equal to the very first middle element.



Search 5

O(1)

$m = \frac{L+R}{2}$

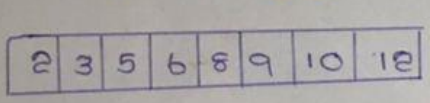
$= \frac{0+7}{2}$

$= 3.5 \approx 3$

Arr[3] == 5 Yes

02. Worst Case

Search element is present at either begining of array or end of the array



Search 2.

1st

$m = \frac{L+R}{2}$

$= \frac{0+7}{2}$

$= 3$

Array[3] == 2 NO

Array[3] > 2 Yes

L = 0

H = m-1 → 2

2nd

$m = \frac{L+R}{2}$

$= \frac{0+2}{2}$

$= 1$

Array[1] == 2 NO

Array[1] > 2 Yes

L = 0

H = m-1 → 0

3rd

$m = \frac{0+0}{2} = 0$

m[0] == 2 Yes

Found

$N/2^k = 1$

N = Array size

k = no of ittoration

01. $N/2^k \rightarrow 8/2^1 = 4$ elm

02 $N/2^k \rightarrow 8/2^2 = 2$ elm

03 $N/2^k \rightarrow 8/2^3 = 1$ elm

OGO TECHNOLOGY
For More Explanation
075 – 930 70 59

03. Avarage Case

$N/2^k = 1$

$N = 2^k$

$\log N = k \log_e 2$

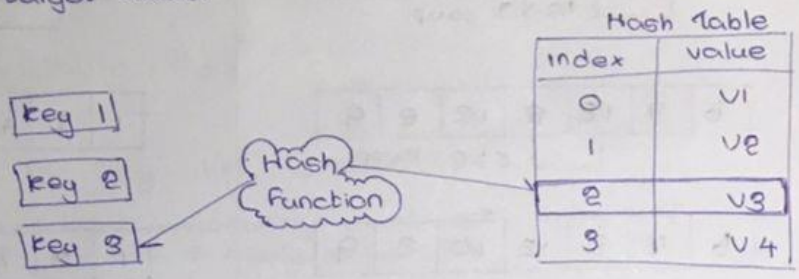
$$k = \log_2 N$$

---

03. Hash Searching Algorithm

The fastest searching algorithm but its also ee most complex to implemant

Hash search algo works by converting the target value into a hash value, which is a unique identifier for target value.

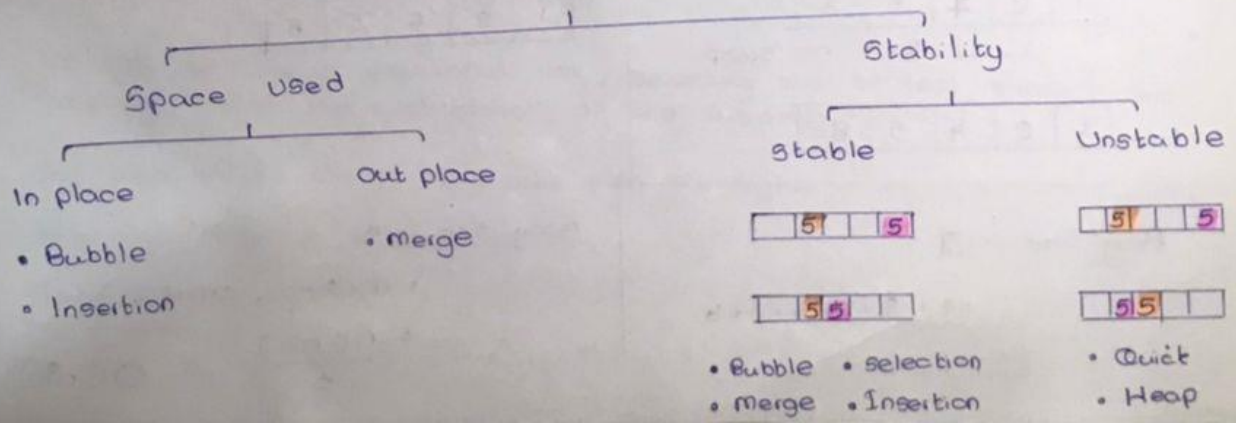


04. Jump Seaching Algorithm

05. Interpolation Algorithm

06. Exponential Searching Algorithm.

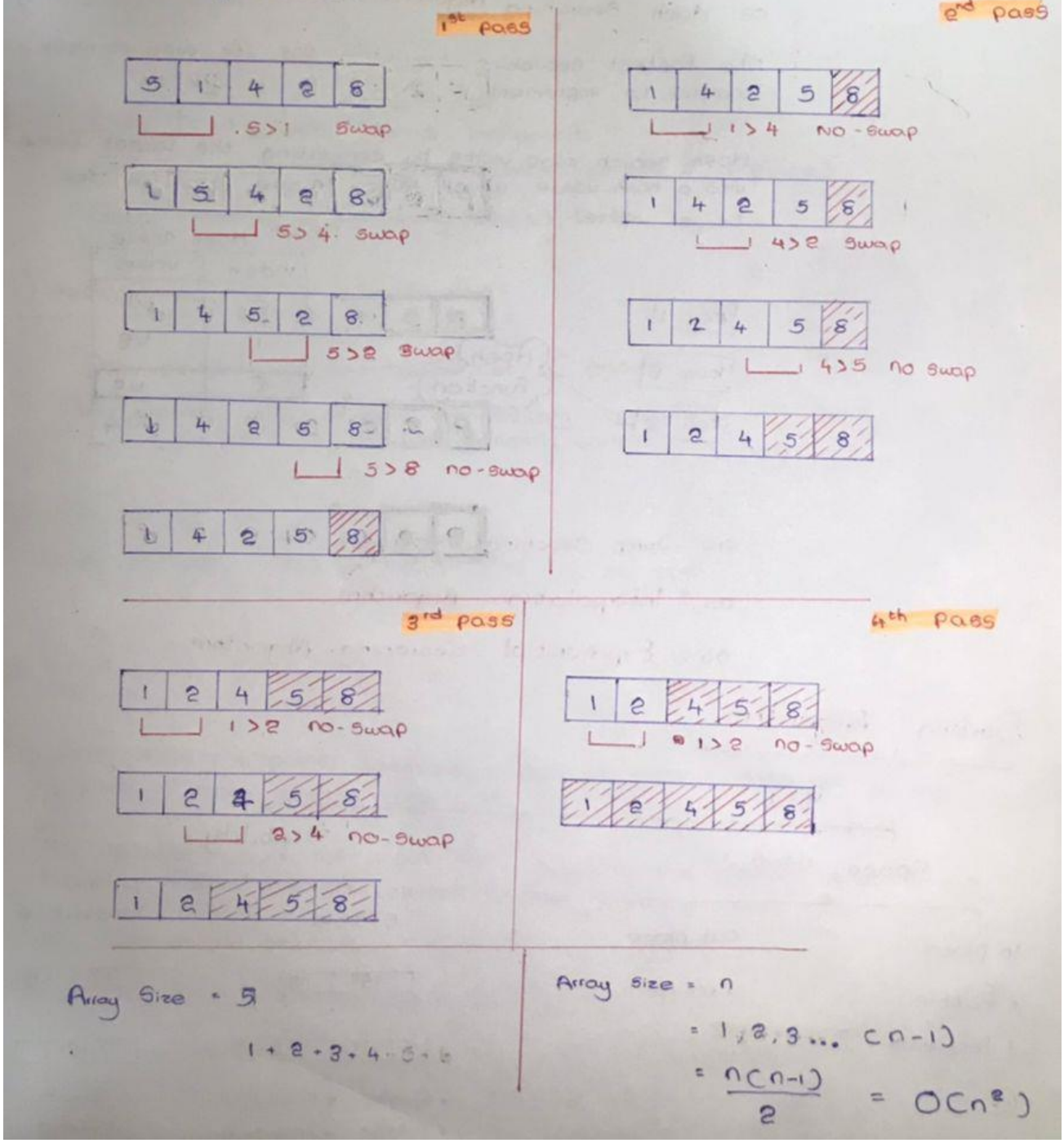
Sorting Algorithm



OGO TECHNOLOGY
For More Explanation
075 – 930 70 59

1. Bubble Sort Algo

- Simple sorting algorithm that works by repeatedly comparing adjacent element in list and swapping them if they are in the wrong oder.
- Pushing larger number to end repeatedly (Right to Left)
- Its best suited for small to moderately arrays ant is not very efficient for larger dataset.



```
Array = [5, 1, 4, 2, 8]

for (i in range(len(array)):

    for (k in range(len(array)-1)):
        if (array[k] > array[k+1]):
            array[k], array[k+1] = array[k+1], array[k]

print(Array)
```

Advantage of Bubble Sort

Easy to Understand and Implement
Sort array/list without additional memory (in place)
Good Performance for nearly sorted lists.

Disadvantage of Bubble Sort

Poor performance for large list ($O(n^2)$)
Inefficient - makes many unnecessary comparisons and swap even the list is nearly sorted

Ex

Web browsers - sort to list of tabs that are open.

02. Selection Sort Algorithm

simple sorting algorithm commonly used for small data set or as a part of more complex algorithm.

Its iterates trough the input list, selecting the smallest element and swapping it with the first element of the subarray

This algo sorted the array left side to right.

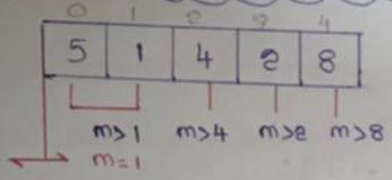
This algo is a in place algorithm

Time Complexity

$$n \longrightarrow (n-1) \dots 2+1 = \frac{n(n-1)}{2} = O(n^2)$$

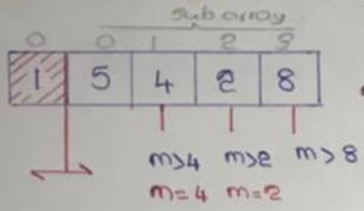
min = m = arr[0] When Start Compair in all subarrays.

Sorted

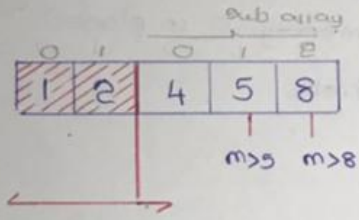


Unsorted

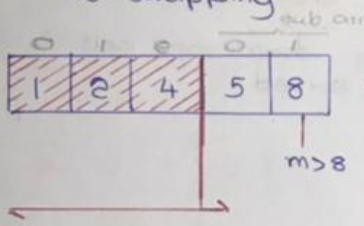
swap 1st element and min element (value 1)



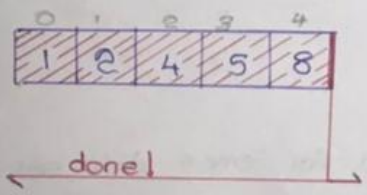
swap 1st element and min element (value 2)



no swapping



no swapping




```
arr = [5, 1, 4, 2, 8]
for i in range(len(arr)):
    min_index = i

    for k in range(i+1, len(arr)):
        if arr[min_index] > arr[k]:
            min_index = k

arr[min_index], arr[k] = arr[k], arr[min_index]
```

Advantage of Seletion sort Algo

- Simple to implement and understand
- Stable (preserves the relative oder of equal element
- In place (no need additional memory)

Disadvantage of Selection sort Algo

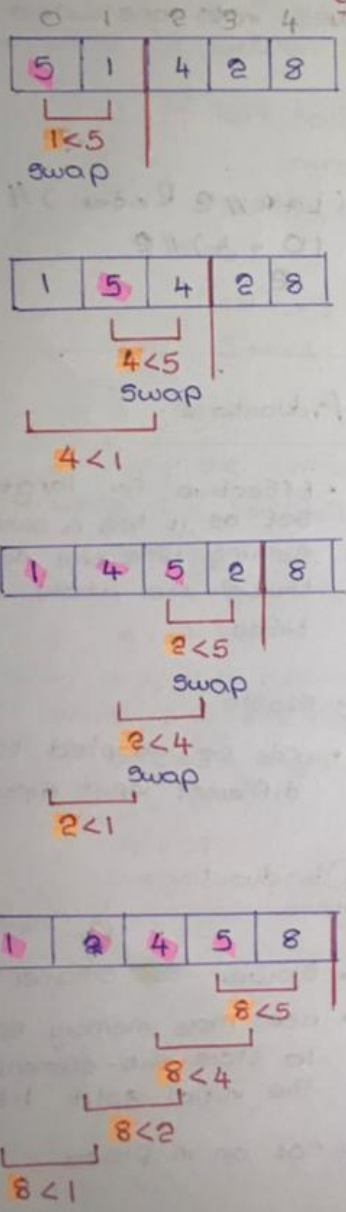
- Inefficient .

03. Insertion Sorting Array

Simple sort algorithm that repeatedly step by step through the list, compaires each elements to its neigbors in left sid and swap them if it is in the wrong oder.

This algo sorted array left to right.

Sorted | Unsorted



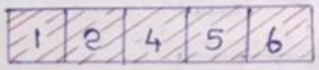
Advantage

- simple to implement and understand
- Stable (preserve the relative order of equal element)
- In-place

Disadvantage

- Inefficient especially for large list

```
for i in range(1, len(arr)):
    key = arr[i]
    j = i-1
    while (j>=0 and key< arr[j]):
        arr[j+1] = arr[j]
        j = j-1
    arr[j+1] = key
```



Time Complexity

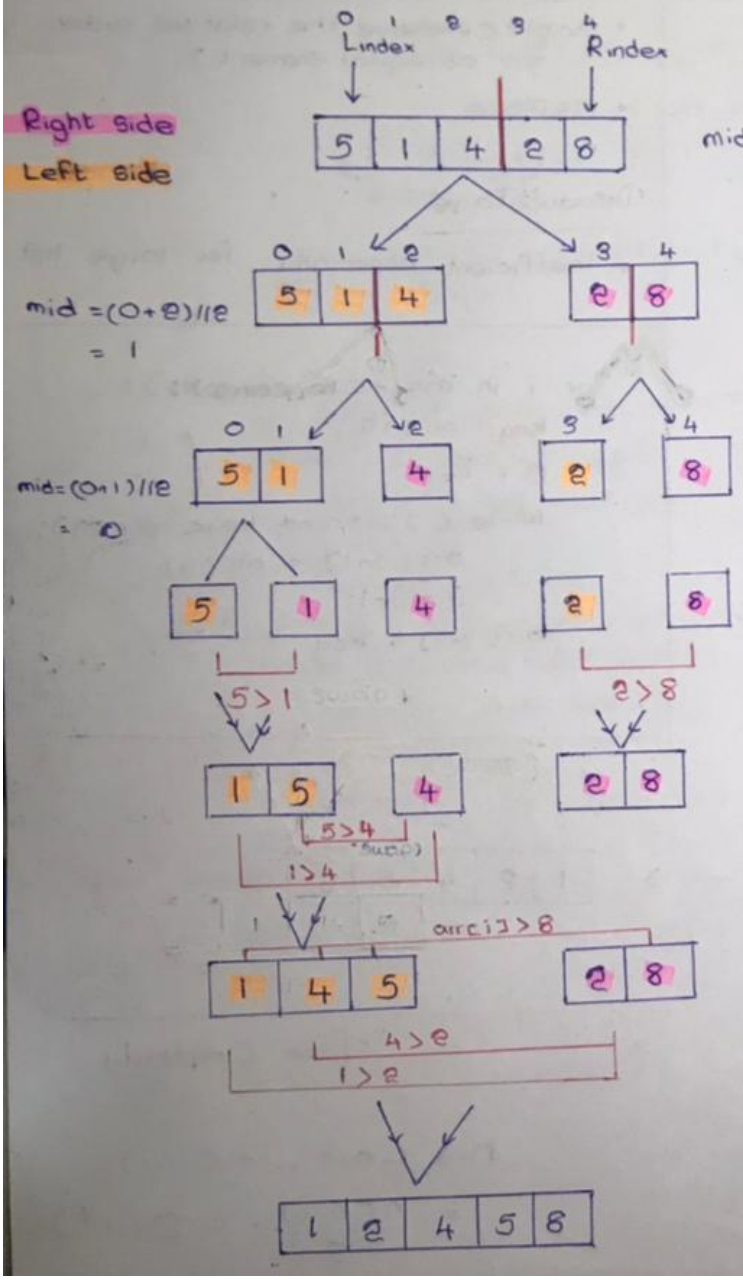
$n \rightarrow 1+2+3 \ldots + (n-1)$

$= \frac{n(n-1)}{2} = O(n^2)$

04. Merge Sort Algorithm

Merge sort is a sorting algorithm that works by dividing an array in to sub arrays, sort each subarray and them merging the sorted subarrays back together to from the final sorted array

01. split array in half
02. call merge sort on each half to sort them recursively
03. merge both sorted halves in to one sorted array.



Advantage

- Effective for larger data set as it has a consistent running time and dose not go through the whole list several times.
- Stable
- Can be adapted to handle different input distribution.

Disadvantage

- Slower for smaller dataset
- uses more memory space to store sub-elements of the initial split list.
- not an in-place.

```
def merge_      (arr):
    if (len(arr) > 1):
        mid = len(arr)//2
        left_half = arr[:mid]
        Right_half = arr[mid:]

        merge_sort(left_half)
        merge_sort(Right_half)

        i = j = k = 0
        # merge the sorted half back together
        while i < len(left_half) and j < len(Right_half):

            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1

            k += 1
        # copy the remaining elements of left_half (if any)
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1
        # copy the remaining elements of righ_half (if any)
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1

    return arr
```

split array in half

Recursively sort the left and right half

```
class Queue:

    def __init__(self):
        self.list = []

    def __str__(self):
        return str(self.list)

    def isEmpty(self):
        if self.list == []
            retun true
        else:
            return false

    def enqueue(self, value):
        return self.list.append(value)

    def dequeue(self, value):
        if self.list.isEmpty():
            return "Queue is empty"
        else:
            return self.list.pop(0)

    def peek(self):
        if self.list.isEmpty():
            return "Queue is empty"
        else:
            return self.list[0]

    def delete(self):
        self.list = none
```

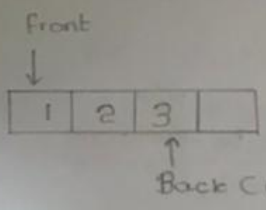
Call to functions

```
myqueue = Queue()

myqueue.enqueue(1)
myqueue.enqueue(2)
myqueue.enqueue(3)

myqueue.isEmpty()
myqueue.dequeue()
myqueue.peek()
myqueue.delete()
```

08 Circular Queue



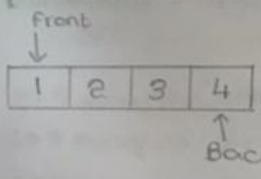
Variation of the standard queue data structure with circular arrangement of a fixed-size array

In a standard queue, when element dequeued, the space is not reclaimed leading to the potential wasted memory.

A circular Queue queue address this issue by reusing the space of dequeued elements.

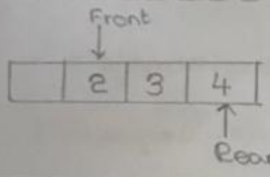
---

enqueue(4)



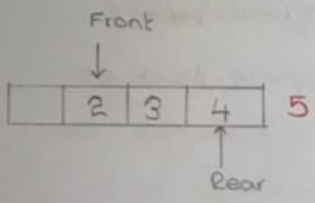
---

dequeue()



---

enqueue(5)

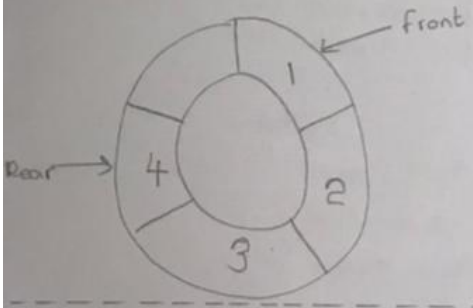


Error

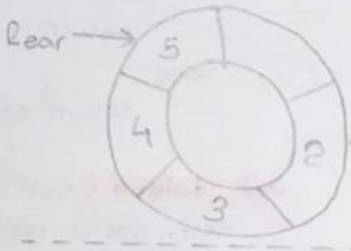
becose queue is full

Queue size is 4, so that we can not add 5th value for queue

There for introducing Circular Queue

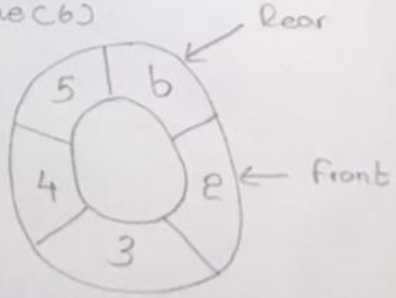


dequeue()

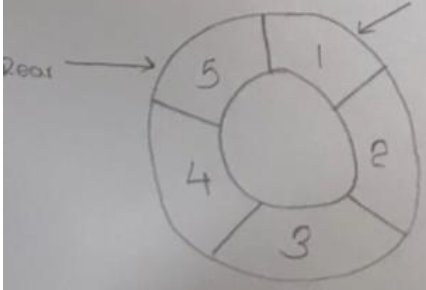


---

enqueue(6)



Front enqueue(5)



```
Class circularqueue:

    def __init__(self, size):
        self.size = size
        self.queue = [none] * size
        self.front = self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue is full")
        elif self.front == -1:
            self.front = self.rear = 0
            self.queue[self.rear] = item
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item

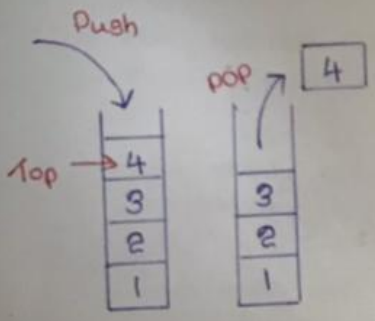
def dequeue(self):
    if self.front == -1:
        print("Queue is empty")
    elif self.front == self.rear:
        item = self.queue[self.front]
        self.front = self.rear = -1
        return item.
    else:
        Item = self.queue[self.front]
        Self.front = (self.front + 1) % self.size
        return item.
```

Call to the function.

```
cq = circularqueue(5)
cq.enqueue(1)
cq.enqueue(2)
:
cq.enqueue(5)

cq.dequeue()
```

04. Stack Data Structure.



A stack is a fundamental data structure that follow LIFO principle

Elements are accessed in a sequantly manner without direct access to elements in the midle.

Used in undo machanism in software application
Used in back button in web browser.

Advantage

- Simplicity and Efficiency
- Memory Management

Disadvantage

- Limited Access
- Dynamic Resizing Overhead
- ...

```
class stack:

    def __init__(self):
        self.stacklist = []

    def __str__(self):
        return self.stacklist

    def isEmpty(self):
        if self.stacklist == []:
            return true
        else:
            return false

    def push(self, value):
        return self.stacklist.append(value)

    def pop(self):
        if self.isEmpty():
            return "stack is Empty"
        else:
            return self.stacklist.pop()

    def peek(self):
        if self.stacklist==[]:
            return "stack is Empty"
        else:
            return self.stacklist[-1]

    def delete(self):
        self.stacklist = none
```

Call to the functions

```
st= stack()

st.push(5)
st.pop()
st.peek()
st.isEmpty()
st.delete()
```

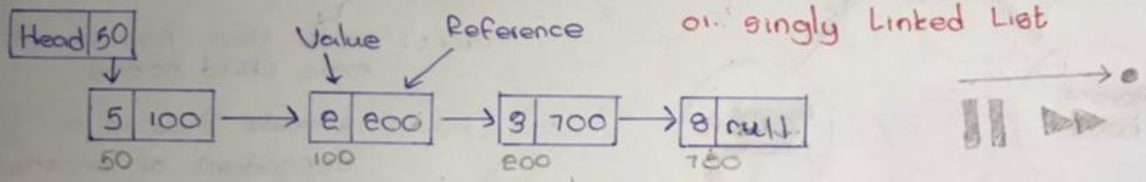
05. Linked Data Structure

link data structure is a data structure which consist of a set of data recodes (node) linked together and organizing by references (link or point)

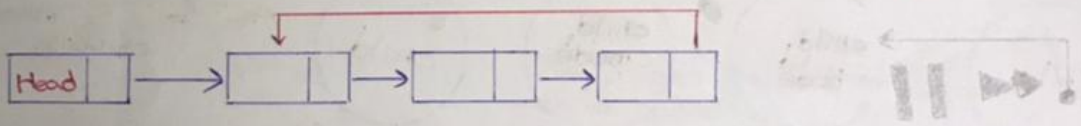
Can easily grow or ~~shrink~~ shrink in size during program execution. as memory allocated dynamically.

Each node can be anywhere in memory.

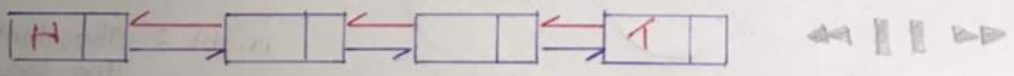
Linked List Types



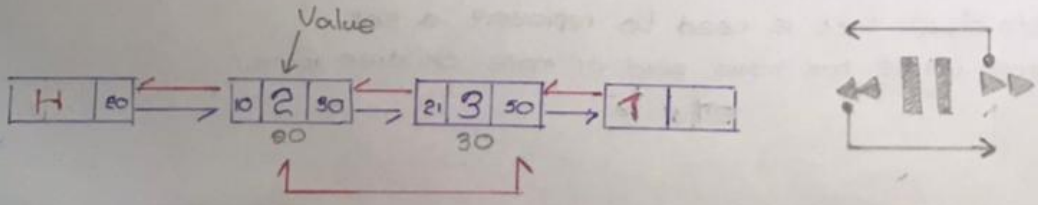
01. singly Linked List



02. Circular Singly Linked list



03. Doubly Linked List



04. Circular Doubly Linked List

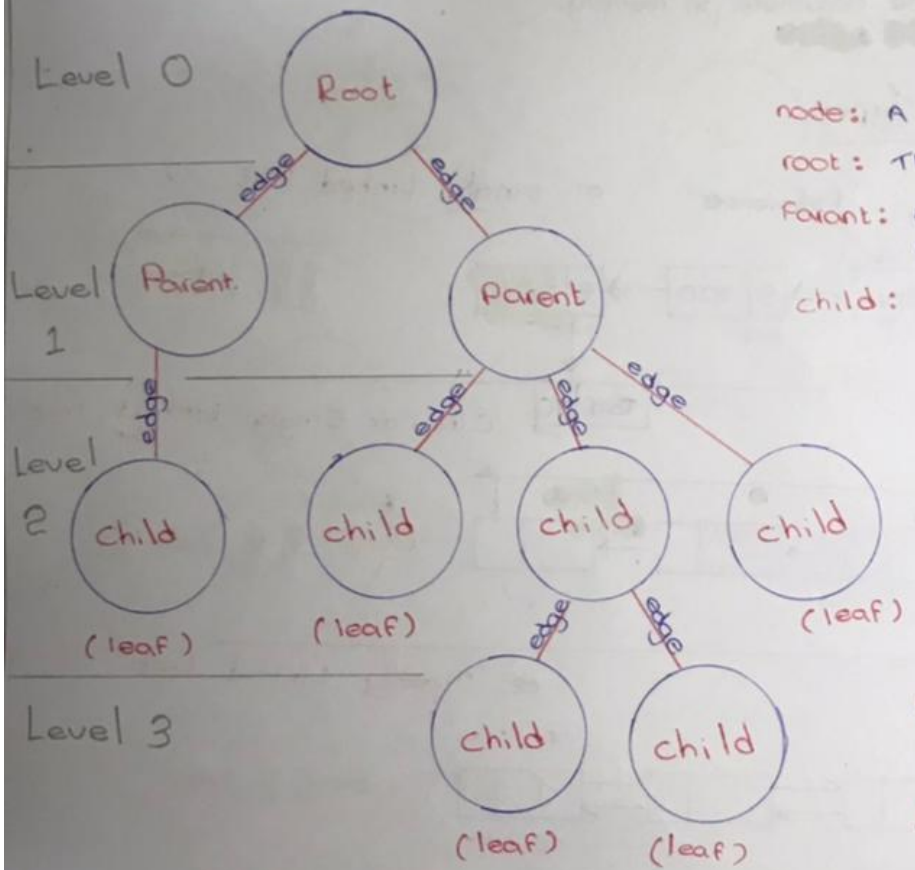
Advantage

- Dynamic memory allocation
- Efficient Ingertion / Deletion
- No- pre allocation of memory

Disadvantage

- Sequential Access
- Memory Overhead
- Traversal Overhead
- Cache locality.

ob. Tree Data Structure



node: A fundamental unit of tree

root: The topmost node

Parent: Node that has one or more child nodes

child: A node that is a dependant of another node.

leaf: node without children.

Sibling: node with the same parent

Hight: lenth of the longest path to root

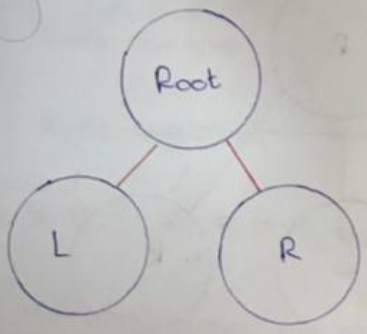
Depth: no of edges root to leaf

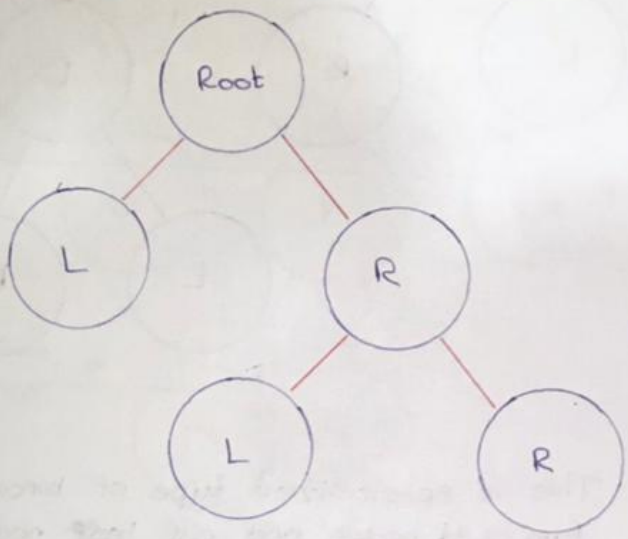
A tree data structure is a hierarchical data structure that is used to represent a set of items where can have zero or more children items.

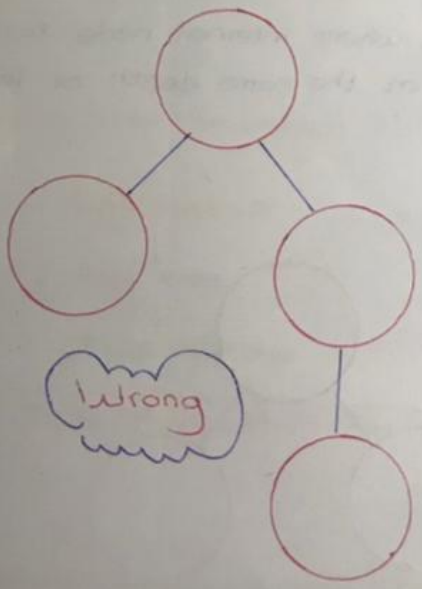
07 Binary Tree Data Structure

Hierarchical data structure composed, where each node has at most two child nodes.

01. Full Binary Tree Data Structure.

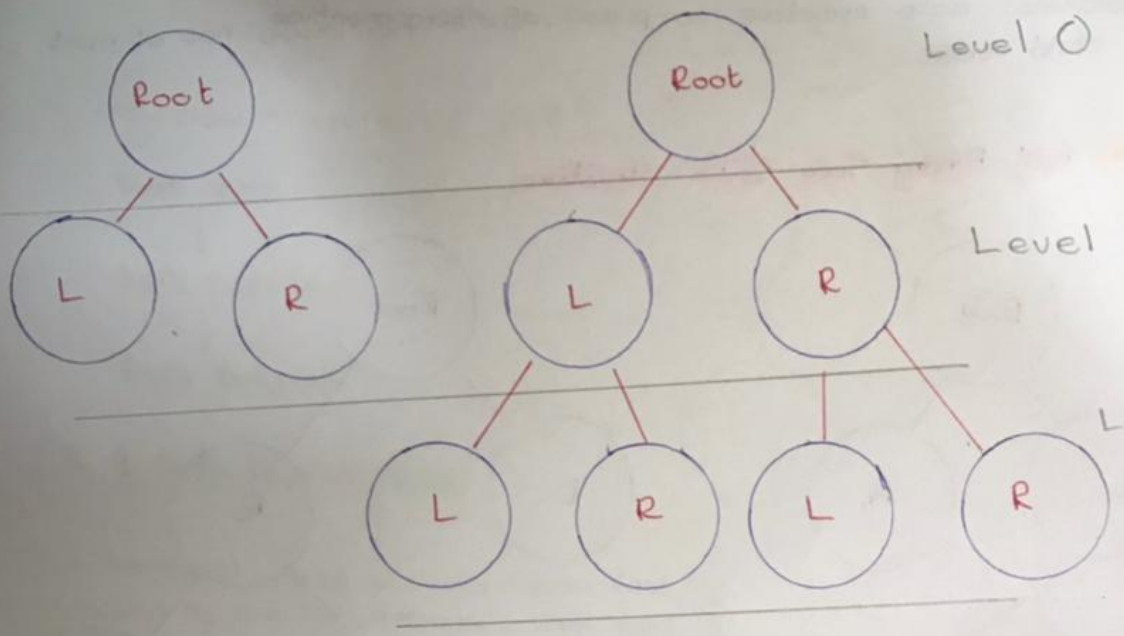






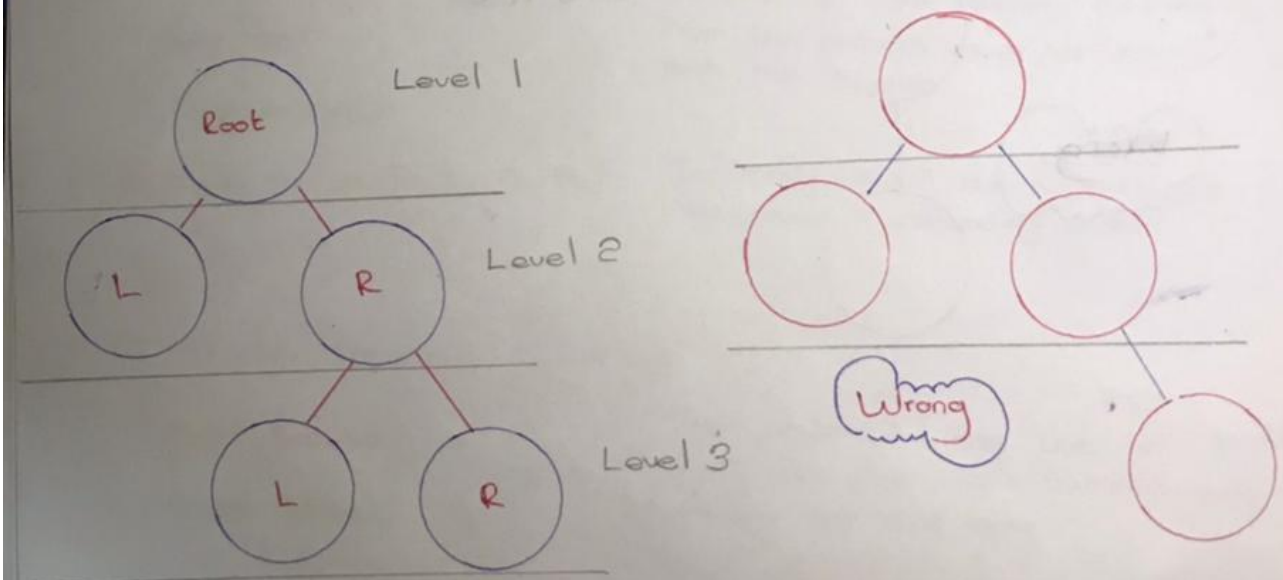
Every node has either zero or two child nodes

02. Perfect Binary Tree



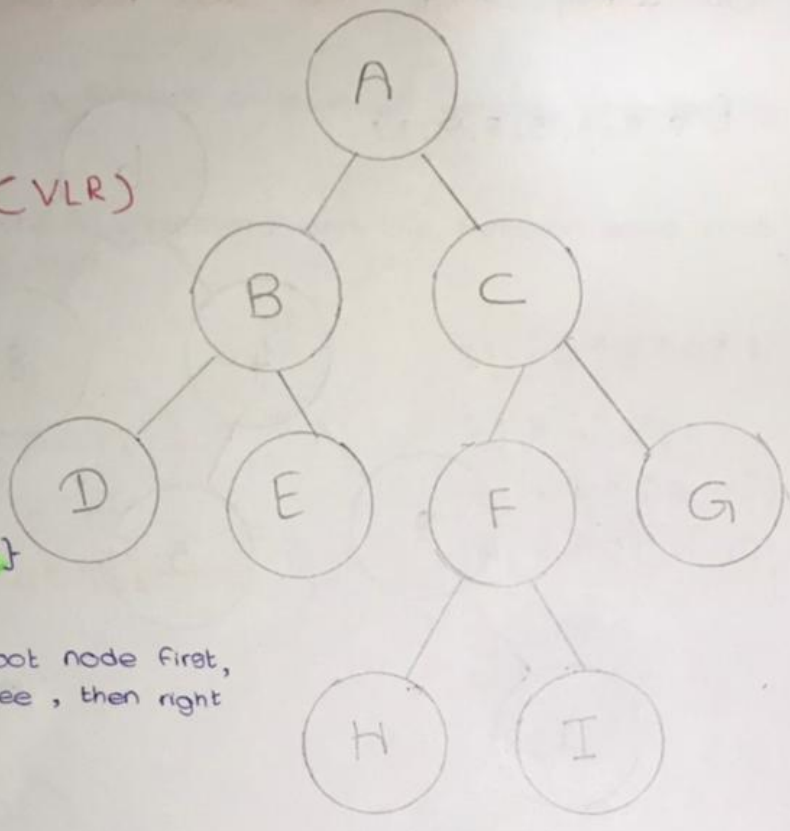
This is speciallized type of binary tree where internal node has two child nodes and all leaf nodes are at the same depth or level

03. Complete Binary Tree



A complet binary tree is special type of binary tree where every level is completly filled, except possibility the last level, which is filled from left to right

Tree Traversal



01. pre oder Traversal (VLR)

Root node
↓
Left subtree
↓
Right subtree

{A, B, D, E, C, F, H, I, G}

This traversal visits the root node first, followed by the left subtree, then right subtree.

02. In oder Traversal (LVR)

Left subtree
↓
Root node
↓
Right subtree

{D, B, E, A, H, F, I, C, G}

This traversal visits the nodes in the oder of their values, starting from the smallest value and ending with the largest

In binary search tree, in produces the values in oecending order.

03 Post oder Traversal (LRV)

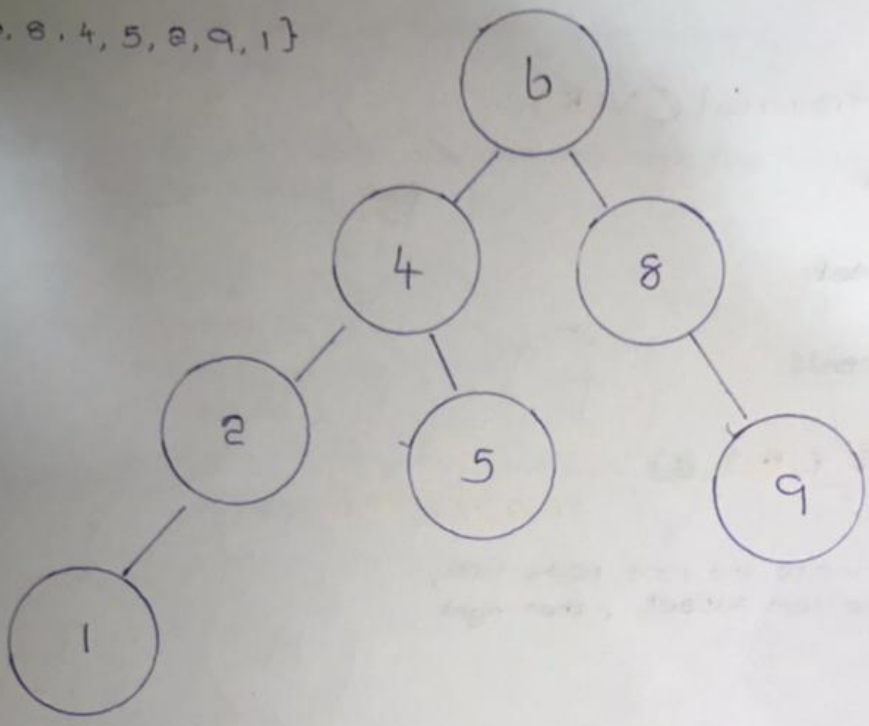
Left subtree
↓
Right subtree
↓
Root node

{D, E, B, H, I, F, G, C, A}

This traversal visits the left subtree first then the right subtree and finally the root node

08 Binary Search Tree Data Structure.

{ 6, 8, 4, 5, 2, 9, 1 }



Each node has at most two child nodes and the data stored in each node is grater than all the data in its left subtree and less than all the data in its right subtree

This property allows for efficient searching, insertion and deletion oparations.

Iteration

Iteration is the process of repeatedly executing a set of statments or block of code until a specific condition is met

its typically uses variables and, increment / decrement and condition

```
for i in range(5):
    print(i)
```

Recursions

The process of in which a function calls itself directly or inderectly to solve a problem.

A problem is divided into sub problems and the function solve each sub problem by calling itself.

```
def fact(n):
    if n==0 or n==1:
        return 1
    else:
        return n* fact(n-1)
```

$4! = 4*3*2*1$

$4! = 4*3!$

$4! = 4*(4-1)!$

$n! = n*(n-1)!$

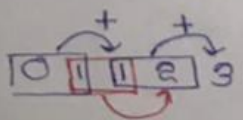
```
result = 1
for i in range(1, n+1):
    result *= i
return result
```

Using Iteration.

Fibonacci Number System

The fibonacci sequence is a siries of numbers in which each numbe is the two preceding once starting from 0 and 1.

So the sequence goes 0, 1, 1, 2, 3, 5... and so on



Leonardo Pisano Bogollo

$F(n) = F(n-1) + F(n-2)$

```
def fib(i):
    if i <= 1:
        return i
    else:
        return fib(i-1) + (fib(i-2)
```

```
for i in range(10):
    print(fib(i), end=",")
```

⟶ 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

Greatest Common Divisor (GCD)

The largest positive integer that divideg both of numbers without leaving remainder.

Its also kwon as Highest common factor.

Euclidean Algorithm

```
def GCD(num1, num2):
    if num2 == 0:
        return num1
    else:
        return GCD(num2, num1 % num2)
```

12 → 1, 2, 3, 4, 6, 12

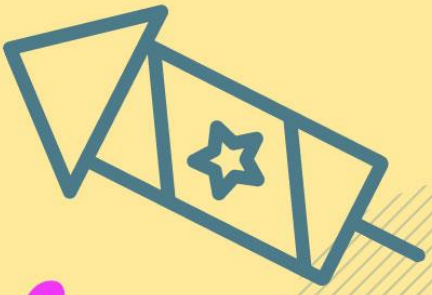
14 → 1, 2, 7, 14

1, 2

1 > 2 (which is Greatest)

GCD = 2





Good Luck

on Your

Exam

My Good Wishes will
always be with you!

From,

OGO TECHNOLOGY





www.BigWishes.net

OGO TECHNOLOGY
For More Explanation
075 – 930 70 59